AGRICULTURE MANAGEMENT SYSTEM

Database Management System Project

Course Name:

Database Management Systems (DBMS)

Group Members

Name	SAP ID
Abdul Rehman	70145826
Arsal Qadir	

Main GitHub Repository:

[https://github.com/your-repo-link]

1. Project Scenario Description

The Farm Management System is a comprehensive database-driven application designed to digitize and manage all operations of an agricultural enterprise. In today's modern agriculture, managing multiple farms, crops, irrigation systems, fertilizers, and sales records manually is inefficient and error-prone. This system solves that problem by storing, organizing, and retrieving all farm-related data in a structured relational database.

The system covers the complete agricultural lifecycle — from registering farmers and their farms, planning crop plantations, managing irrigation schedules and fertilizer applications, recording harvests, and finally tracking crop sales. It also includes a disease tracking module so that farmers can record and monitor crop diseases affecting their plantations.

Real-World Use Case:

- A farmer registers himself and adds all his farms.
- He plans which crops to plant on which farm (Plantation).
- He schedules irrigation for each plantation.
- He applies different fertilizers to crops (Many-to-Many relationship).
- After harvest, he records yields and then logs sales to buyers.
- If a disease appears, it is recorded against the affected crops

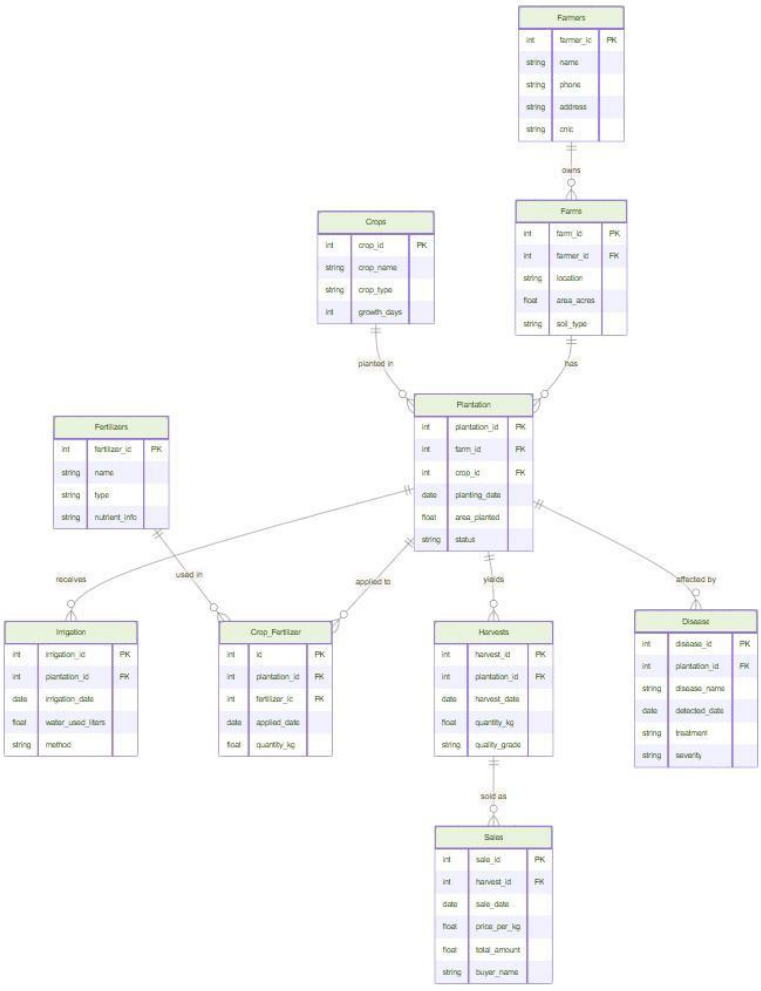
2.1 List of Tables

#	Table Name	Description	Key Relationships
1	Farmers	Stores farmer personal information	PK: farmer_id
2	Farms	Each farmer can own multiple farms	FK: farmer_id → Farmers
3	Crops	All crop types registered in the system	PK: crop_id
4	Plantation	Records which crop is planted on which farm	FK: farm_id, crop_id
5	Irrigation	Irrigation schedule per plantation	FK: plantation_id
6	Fertilizers	Fertilizer types available	PK: fertilizer_id
7	Crop_Fertilizer	M:N bridge between Crops & Fertilizers	FK: crop_id, fertilizer_id
8	Harvests	Harvest records per plantation	FK: plantation_id
9	Sales	Sales records per harvest	FK: harvest_id
10	Disease	Diseases recorded per crop	FK: crop_id

2.2 Relationships Summary

Parent Table	Relationship	Child Table	Type
Farmers	1 : M	Farms	One-to-Many
Farms	1 : M	Plantation	One-to-Many
Crops	1 : M	Plantation	One-to-Many
Crops	M : N	Fertilizers (via Crop_Fertilizer)	Many-to-Many
Plantation	1 : M	Irrigation	One-to-Many
Plantation	1 : M	Harvests	One-to-Many
Harvests	1 : M	Sales	One-to-Many
Crops	1 : M	Disease	One-to-Many

3. Entity-Relationship Diagram



4. Description of the 10 Features

Each feature represents a real-world database operation implemented through SQL queries. Below is a detailed description of all 10 features:

Feature 1: Register a New Farmer:

Description: This feature allows adding a new farmer into the system with their name, contact number, and address. It is the first step before any farm or crop data can be entered.

- Real-World Use: When a new farmer joins the cooperative, their profile is created.

Feature 2: Add a New Farm for a Farmer

Description: Registers a farm under a specific farmer. Each farm has a location, size in acres, and a soil type.

- Real-World Use: A farmer buys a new piece of land and registers it.

Feature 3: Record a New Crop Plantation

Description: Records which crop is being planted on which farm, along with the planting date and expected harvest date.

- Real-World Use: Planting season begins and farmer logs which crop goes where.

Feature 4: View All Crops on a Specific

Description: Retrieves all plantations on a given farm using JOIN between Farms, Plantation, and Crops tables. Shows crop name, planting date, and expected harvest date.

- Real-World Use: Farm manager checks what is currently planted on Farm No. 3.

Feature 5: Update Irrigation Schedule

Description: Updates the irrigation frequency or last irrigation date for a specific plantation. Useful when the schedule changes due to rainfall or crop growth stage.

- Real-World Use: After heavy rain, irrigation frequency is reduced.

Feature 6: Assign Fertilizer to a Crop

Description: Handles the Many-to-Many relationship between Crops and Fertilizers. A crop can receive multiple fertilizers, and a fertilizer can be used on many crops. This feature inserts a record in the Crop_Fertilizer bridge table.

- Real-World Use: Wheat crop gets assigned Urea and DAP fertilizers.

Feature 7: Record a Harvest

Description: Records the yield of a plantation after harvesting. Stores quantity harvested (in kg or tons), harvest date, and quality grade.

- Real-World Use: After wheat is harvested, the yield data is recorded.

Feature 8: Record a Sale of Harvested Crop

Description: Logs a sale transaction for a specific harvest. Records buyer name, quantity sold, price per unit, and sale date.

- Real-World Use: Farmer sells 500 kg of wheat to a local market buyer.

Feature 9: View Total Sales Revenue Per Farm

Description: A reporting feature that aggregates total sales revenue per farm using JOIN across multiple tables (Sales → Harvests → Plantation → Farms) and GROUP BY.

- Real-World Use: Farm owner wants to see which farm generated the most revenue.

Feature 10: Record and View Crop Diseases

Description: Allows recording of a disease detected on a specific crop, including disease name, detection date, and treatment suggested.

- Real-World Use: Farmer notices blight on tomatoes and logs it for record-keeping.

5. SQL Queries Used for Each Feature

5.1 DDL — CREATE TABLE Statements

Farmers Table

```
CREATE TABLE Farmers (
 Farmer_id INT PRIMARY KEY AUTO_INCREMENT,
 Full_name VARCHAR(100) NOT NULL,
 Contact_no VARCHAR(15),
 Address VARCHAR(255),
 Created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Farms Table

```
CREATE TABLE Farms (
 Farm_id INT PRIMARY KEY AUTO_INCREMENT,
 Farmer_id INT NOT NULL,
 Farm_name VARCHAR(100),
 Location VARCHAR(255),
 Area_acres DECIMAL(10,2),
 Soil_type VARCHAR(50),
 FOREIGN KEY (farmer_id) REFERENCES Farmers(farmer_id)
```

```
 ON DELETE CASCADE
);
```

Crops Table

```
CREATE TABLE Crops (
 Crop_id INT PRIMARY KEY AUTO_INCREMENT,
 Crop_name VARCHAR(100) NOT NULL,
 Season VARCHAR(50),
 Duration_days INT
);
```

Plantation Table

```
CREATE TABLE Plantation (
 Plantation_id INT PRIMARY KEY AUTO_INCREMENT,
 Farm_id INT NOT NULL,
 Crop_id INT NOT NULL,
 Planting_date DATE NOT NULL,
 Expected_harvest DATE,
 Status VARCHAR(30) DEFAULT ‘Active’,
 FOREIGN KEY (farm_id) REFERENCES Farms(farm_id),
 FOREIGN KEY (crop_id) REFERENCES Crops(crop_id)
);
```

Irrigation Table

```
CREATE TABLE Irrigation (
 Irrigation_id INT PRIMARY KEY AUTO_INCREMENT,
Plantation_id INT NOT NULL,
 Method VARCHAR(50),
 Frequency_days INT,
 Last_irrigated DATE,
 Water_source VARCHAR(100),
 FOREIGN KEY (plantation_id) REFERENCES Plantation(plantation_id)
);
```

Fertilizers Table

```
CREATE TABLE Fertilizers (
 Fertilizer_id INT PRIMARY KEY AUTO_INCREMENT,
 Fertilizer_name VARCHAR(100) NOT NULL,
 Type VARCHAR(50),
 Npk_ratio VARCHAR(30)
);
```

Crop_Fertilizer Table (M:N Bridge)

```
CREATE TABLE Crop_Fertilizer (
```

```
 Cf_id INT PRIMARY KEY AUTO_INCREMENT,
 Crop_id INT NOT NULL,
 Fertilizer_id INT NOT NULL,
 Application_date DATE,
 Quantity_kg DECIMAL(8,2),
 FOREIGN KEY (crop_id) REFERENCES Crops(crop_id),
 FOREIGN KEY (fertilizer_id) REFERENCES Fertilizers(fertilizer_id)
);
```

Harvests Table

```
CREATE TABLE Harvests (
 Harvest_id INT PRIMARY KEY AUTO_INCREMENT,
 Plantation_id INT NOT NULL,
 Harvest_date DATE NOT NULL,
 Quantity_kg DECIMAL(10,2),
 Quality_grade VARCHAR(10),
 Notes TEXT,
 FOREIGN KEY (plantation_id) REFERENCES Plantation(plantation_id)
);
```

Sales Table

```
CREATE TABLE Sales (
```

```
 Sale_id INT PRIMARY KEY AUTO_INCREMENT,
 Harvest_id INT NOT NULL,
 Buyer_name VARCHAR(100),
 Quantity_sold_kg DECIMAL(10,2),
 Price_per_kg DECIMAL(8,2),
 Sale_date DATE,
 Total_amount DECIMAL(12,2) GENERATED ALWAYS AS
 (quantity_sold_kg * price_per_kg) STORED,
 FOREIGN KEY (harvest_id) REFERENCES Harvests(harvest_id)
```

Disease Table

```
CREATE TABLE Disease (
 Disease_id INT PRIMARY KEY AUTO_INCREMENT,
 Crop_id INT NOT NULL,
 Disease_name VARCHAR(100),
 Detection_date DATE,
 Severity VARCHAR(20),
 Treatment TEXT,
 FOREIGN KEY (crop_id) REFERENCES Crops(crop_id)
);
```

5.2 DML — INSERT Sample Data

- **Insert Farmers**

INSERT INTO Farmers (full_name, contact_no, address) VALUES

(‘Ali Hassan’, ‘0300-1234567’, ‘Village Ravi, Lahore’),

(‘Sara Khan’, ‘0321-9876543’, ‘Bhalwal, Sargodha’),

(‘Usman Tariq’, ‘0311-5554433’, ‘Chichawatni, Sahiwal’);

- **Insert Farms**

INSERT INTO Farms (farmer_id, farm_name, location, area_acres, soil_type) VALUES

(1, ‘Ali Farm 1’, ‘Ravi Road, Lahore’, 12.5, ‘Clay Loam’),

(1, ‘Ali Farm 2’, ‘Sheikhupura’, 8.0, ‘Sandy’),

(2, ‘Sara Land’, ‘Bhalwal, Sargodha’, 20.0, ‘Alluvial’);

- **Insert Crops**

INSERT INTO Crops (crop_name, season, duration_days) VALUES

(‘Wheat’, ‘Rabi’, 120),

(‘Rice’, ‘Kharif’, 150),

(‘Cotton’, ‘Kharif’, 180),

(‘Tomato’, ‘Rabi’, 90);

- **Insert Plantation**

INSERT INTO Plantation (farm_id, crop_id, planting_date, expected_harvest) VALUES

(1, 1, ‘2025-11-01’, ‘2026-03-01’),

(1, 4, ‘2025-11-15’, ‘2026-02-15’),

(3, 2, ‘2025-06-01’, ‘2025-11-01’);

- **Insert Irrigation**

INSERT INTO Irrigation (plantation_id, method, frequency_days, last_irrigated,

Water_source) VALUES

(1, ‘Drip’, 7, ‘2025-12-01’, ‘Canal’),

(2, ‘Flood’, 5, ‘2025-12-03’, ‘Tubewell’);

- **Insert Fertilizers**

INSERT INTO Fertilizers (fertilizer_name, type, npk_ratio) VALUES

(‘Urea’, ‘Nitrogenous’, ’46-0-0’),

(‘DAP’, ‘Phosphatic’, ’18-46-0’),

(‘SOP’, ‘Potassic’, ‘0-0-50’);

- **Insert Crop_Fertilizer (M:N)**

INSERT INTO Crop_Fertilizer (crop_id, fertilizer_id, application_date, quantity_kg)

VALUES

(1, 1, ‘2025-12-01’, 50.0),

(1, 2, ‘2025-12-15’, 30.0),

(2, 1, ‘2025-07-01’, 40.0);

- **Insert Harvests**

INSERT INTO Harvests (plantation_id, harvest_date, quantity_kg, quality_grade) VALUES

(1, ‘2026-03-05’, 4500.00, ‘A’),

(2, ‘2026-02-20’, 1200.00, ‘B’);

- **Insert Sales**

INSERT INTO Sales (harvest_id, buyer_name, quantity_sold_kg, price_per_kg, sale_date)

VALUES

(1, ‘Ahmed Traders’, 3000.00, 45.00, ‘2026-03-10’),

(2, ‘Local Market’, 800.00, 80.00, ‘2026-02-25’);

- **Insert Disease**

INSERT INTO Disease (crop_id, disease_name, detection_date, severity, treatment)

VALUES

(4, ‘Tomato Blight’, ‘2025-12-20’, ‘High’, ‘Spray Mancozeb fungicide’);

5.3 DML — SELECT Queries (Features)

View All Crops on a Specific Farm

```
SELECT f.farm_name, c.crop_name, p.planting_date,
 p.expected_harvest, p.status
FROM Plantation p
JOIN Farms f ON p.farm_id = f.farm_id
JOIN Crops c ON p.crop_id = c.crop_id
WHERE f.farm_id = 1;
```

Total Sales Revenue Per Farm

```
SELECT f.farm_name,
 SUM(s.total_amount) AS total_revenue,
 SUM(s.quantity_sold_kg) AS total_kg_sold
FROM Sales s
JOIN Harvests h ON s.harvest_id = h.harvest_id
JOIN Plantation p ON h.plantation_id = p.plantation_id
JOIN Farms f ON p.farm_id = f.farm_id
GROUP BY f.farm_id, f.farm_name
ORDER BY total_revenue DESC;
```

View All Diseases on Current Crops

```
SELECT c.crop_name, d.disease_name, d.detection_date,
 d.severity, d.treatment
FROM Disease d
JOIN Crops c ON d.crop_id = c.crop_id
ORDER BY d.detection_date DESC;
```

View Fertilizers Used Per Crop

```
SELECT c.crop_name, f.fertilizer_name, f.npk_ratio,
 Cf.application_date, cf.quantity_kg
FROM Crop_Fertilizer cf
JOIN Crops c ON cf.crop_id = c.crop_id
JOIN Fertilizers f ON cf.fertilizer_id = f.fertilizer_id
ORDER BY c.crop_name;
```

5.4 DML — UPDATE and DELETE

Update Irrigation Frequency

UPDATE Irrigation

SET frequency_days = 10,

Last_irrigated = '2025-12-15'

WHERE irrigation_id = 1;

Delete a Plantation

DELETE FROM Plantation

WHERE plantation_id = 3;

- Note: ON DELETE CASCADE must be set on child tables (Irrigation, Harvests)

Update Crop Status After Harvest

UPDATE Plantation

SET status = 'Harvested'

WHERE plantation_id = 1;

5.5 DCL — Data Control Language

DCL commands control access to the database. These are used to grant or revoke permissions

To users.

- **Create a read-only user for reporting**

CREATE USER ‘farm_viewer’@’localhost’ IDENTIFIED BY ‘ViewPass123’;

- **Grant SELECT permission on all tables**

GRANT SELECT ON farm_management.* TO ‘farm_viewer’@’localhost’;

- **Create a data-entry user**

CREATE USER ‘farm_operator’@’localhost’ IDENTIFIED BY ‘OpPass456’;

- **Grant INSERT and UPDATE only (no DELETE)**

GRANT SELECT, INSERT, UPDATE ON farm_management.* TO ‘farm_operator’@’localhost’;

- **Revoke UPDATE from viewer**

REVOKE UPDATE ON farm_management.* FROM ‘farm_viewer’@’localhost’;

- **Show current grants**

SHOW GRANTS FOR ‘farm_viewer’@’localhost’;

8. Conclusion

The Farm Management System demonstrates the practical use of relational database design in solving real agricultural management challenges. By implementing all 10 functional features using proper DDL, DML, and DCL commands, this project showcases:

- Proper use of Primary Keys and Foreign Keys for data integrity
- Normalization up to 3NF to eliminate redundancy
- All three relationship types: 1:1 (implicit), 1:M, and M:N
- Real-world SQL operations: INSERT, SELECT, UPDATE, DELETE, JOIN, GROUP BY
- Access control using DCL GRANT and REVOKE commands
- A complete agricultural lifecycle from registration to sales